```
Data Sent To Server
Received from server:
hello
Enter Data For the server or press CTRL-D to exit
```

If you have read the previous article in this series, you will know that my client and server are both running on the same Fedora 15 system in two terminal windows. The server listens on the 'port number' 11710 on the loop-back address 127.0.0.1. When the client requests some service from the server, the server processes it and waits for another client or another request from the same client. As before, the service simply returns the string received from the client.

Now let's take simple algorithms for the TCP client and server. First, the client:

```
Get the socket descriptor (using the socket function).
Update a socket address structure with the server's IP
address and port.
Complete TCP's three-way handshake by calling the connect
function.
Get data for sending through the socket and send it over the
socket.
Receive data from the socket (from the server). Here, the
process waits—that is, gets blocked.
Close the socket descriptor.
```

A simple TCP server algorithm:

```
Get the socket descriptor (using the socket function).
Update a socket address structure with the local address and
port number.
Associate (bind) the above address with the socket.
Call the "listen" function.
Repeat forever {
     Get blocked in "accept" call, till it returns a
connection descriptor.
     serve_client (connection descriptor) {
       Receive data from a client (Here the process waits
that is get blocked)
       Process Received data (Specific to the particular
problem)
       Send data to this client.
       If client closes connection,
       then
           close connection descriptor.
           break (continue waiting in accept)
       else
       continue with this function (server_client)

     }
}
```

Now, please refer to the downloaded source code. The *socket* function returns a socket descriptor (IPv4, *SOCK_STREAM (TCP)*) in the variable *sd*. An address structure *struct sockaddr_in* (whose declaration is in the file */usr/include/netinet/in.h* in Linux) is needed; you can get it by including the header file *<netinet/in.h>*. In the client code, you need to populate this address structure (the *serveraddress* variable) with the address family, server IP address, and port number. Note that this needs to be populated in big-endian order, so use functions/macros like *htons* and *inet_addr*. Here, *htons(atoi(argv[2]))* transforms the string the user passes as the command-line argument into a big-endian short integer. Similarly, *inet_addr(argv[1])* transforms the IP address command-line argument into a 32-bit IPv4 address in big-endian order.

Next, call the *connect* function in the client:

```
ret = connect (sd, (struct sockaddr*)&serveraddress, sizeof
(serveraddress));
```

If successful, it means that TCP three-way handshake is completed and you get a connected socket, which can be used to send or receive data. Now you can use the *'read'* and *'write'* functions to read/write data. An important point to be noted here is that, UDP deals with entire datagrams, but TCP is a byte oriented protocol.This requires the receiver to understand how much data to read. But this information should come from the sender. Or it can so happen that the sender sends some data, which the receiver reads and then waits to receive more data. Now if the sender closes the connection, the "read" function will return a value of 0, in the receiver's side. This might be used by the receiver to know that the peer has closed the connection. But once a connection is closed, the sender will not be able to send any more data. It again will have to initiate connection request. This may be undesirable, but this is the feature of TCP. The code below tries to read *'n'* (passed as argument) bytes of data from the socket descriptor *'sd'* to storage (address of a variable or name of an array).

```
int myread (int sd, int n, void* buffer) {
        int ret = 0;
        int pointer = 0;
        int totnoofbytes = 0;
        while (1) {
                ret = read (sd, buffer + pointer, n -
totnoofbytes);
                if (0 > ret) {
                        perror ("I am getting error in
read!!\n");
                        return -1;
                }
                totnoofbytes = totnoofbytes + ret;
                pointer = pointer + ret;
                //totnoofbytes = 0 means, nothing more to
```